

REST API

Практический вебинар для начинающих разработчиков

90 минут · Язык-агностик · Уровень: Junior

Структура вебинара

Вебинар состоит из 8 блоков. Каждый блок начинается с «боли» — реальной проблемы, с которой сталкивается джун — и затем объясняет, как REST её решает.

#	Тема	Время	Ключевой тезис
0	Вступление	5 мин	Зачем этот вебинар
1	Зачем вообще API?	10 мин	REST — архитектурный стиль, не протокол
2	Ресурсы и URL	15 мин	URL — существительное, не глагол
3	HTTP методы и статус-коды	15 мин	Каждый метод — это обещание
4	Запросы и ответы	10 мин	Консистентность и структура
5	Безопасность	15 мин	CORS, API Key, JWT — обязательный минимум
6	Версионирование	5 мин	Не сломай клиентов
7	Практика / Демо	10 мин	Разбираем реальный API
8	Итог и Q&A	5 мин	Чеклист хорошего API

Блок 0 — Вступление

5 минут

Что сказать

Начни с простого вопроса аудитории: «Кто из вас уже делал `fetch()` или `axios`-запрос в своём проекте?» Почти все поднимут руку. Отлично — значит, вы уже работаете с API, даже если не знаете всех правил.

Объясни структуру вебинара: каждый блок начинается с боли — реального сценария из жизни джуна — и затем мы вместе разбираем, как правильно это делать.

Зачем этот вебинар

- REST API — это язык, на котором разговаривают все современные сервисы
- Джун сталкивается с ним в первую же неделю на любом проекте
- Большинство проблем в интеграциях — это нарушение базовых принципов REST
- После этого вебинара у вас будет чёткая картина: что правильно, что нет и почему

Roadmap

Покажи структуру вебинара на экране. Скажи: «Мы пройдем от самых основ — зачем вообще нужен API — до конкретных практических вещей: безопасности, версионирования и разбора реального API.»

Блок 1 — Зачем вообще API?

10 минут

Боль: хаос без стандартов

Открой с вопроса: «Представь, что у тебя есть мобильное приложение, веб-сайт и партнёрский сервис — и все трое хотят работать с одними и теми же данными пользователей. Как их подключить?»

Можно писать отдельную логику для каждого. Можно дать прямой доступ к базе данных — и это реально делали когда-то. Можно делать бинарные RPC протоколы с кучей boilerplate. Всё это — путь к боли.

Исторический контекст: в нулевых доминировал SOAP — XML поверх HTTP с жёсткими схемами, WSDL, конвертами... Это работало, но разработка была мучительной. REST появился как ответ на эту сложность.

Концепция: API как контракт

API (Application Programming Interface) — это контракт между клиентом и сервером. Сервер говорит: «Если ты пришлёшь мне вот это — я отвечу вот этим». Клиент не знает, что внутри сервера. Сервер не знает, кто клиент. Им не нужно.

Что такое REST

REST (Representational State Transfer) — это не библиотека и не протокол. Это набор архитектурных ограничений, которые описал Рой Филдинг в своей диссертации в 2000 году.

Ключевое слово — ограничения. REST говорит: «Если ты будешь следовать этим правилам — твой API будет предсказуемым, масштабируемым и простым в использовании».

Почему REST победил

- Работает поверх обычного HTTP — не нужен специальный транспорт
- Читаемые URL — человек может понять, что делает запрос
- Легко тестировать — достаточно curl или Postman
- Огромная экосистема инструментов и документации
- Низкий порог входа для клиентов

Совет: Скажи аудитории: «REST — это скорее соглашение, чем спецификация. Нет жёсткого стандарта, который запрещает делать неправильно. Поэтому в реальности встречаются API, которые называют себя REST, но нарушают базовые принципы.»

Блок 2 — Ресурсы и URL

15 минут

Боль: глагольные URL

Покажи аудитории такой набор эндпоинтов:

```
POST /getUser
POST /createNewUser
POST /deleteUserById?id=42
POST /getUserOrders
```

Спроси: «Что не так?» Скорее всего кто-то скажет «всё POST» или «слишком длинные названия». Правильный ответ: URL содержат глаголы, хотя это должно делать HTTP-метод. И всё через POST, хотя это GET, DELETE и т.д.

Это не выдуманный пример. Такое встречается в реальных проектах — особенно когда API проектировал разработчик, который раньше писал только SQL-хранимые процедуры.

Концепция: ресурс — это существительное

В REST URL описывает ресурс — некую сущность или коллекцию сущностей. Не действие, не операцию. Ресурс.

Действие описывает HTTP-метод. Вместе они дают полную картину:

```
Хорошо: GET /users — получить список пользователей
Хорошо: POST /users — создать пользователя
Хорошо: GET /users/42 — получить пользователя с id=42
Хорошо: DELETE /users/42 — удалить пользователя с id=42
```

Правила именования

- Используй существительные во множественном числе: /users, /orders, /products
- Нижний регистр, слова через дефис: /user-profiles (не /userProfiles, не /UserProfiles)
- Никаких глаголов в URL: не /getUser, не /createOrder
- Никаких расширений файлов: не /users.json

Вложенность ресурсов

Когда один ресурс логически принадлежит другому — отражай это в URL:

```
GET /users/42/orders — все заказы пользователя 42
GET /users/42/orders/7 — заказ №7 пользователя 42
POST /users/42/orders — создать заказ для пользователя 42
```

Частая ошибка: Не уходи глубже двух уровней вложенности.
/users/42/orders/7/items/3/reviews — это уже больно читать и поддерживать. Если нужно глубже — рассмотри отдельный эндпоинт для вложенного ресурса: /reviews/15.

Query параметры vs Path параметры

Path параметр (/users/42) — когда ты обращаешься к конкретному ресурсу по идентификатору.

Query параметр (/users?role=admin&active=true) — для фильтрации, поиска, сортировки и пагинации. Это опциональные модификаторы, не часть адреса ресурса.

Хорошо: GET /users/42 — конкретный пользователь

Хорошо: GET /users?role=admin — фильтрация списка

Хорошо: GET /products?page=2&limit=20 — пагинация

Плохо: GET /users?id=42 — id через query, хотя это прямой адрес

Блок 3 — HTTP методы и статус-коды

15 минут

Боль: POST для всего

Джун делает форму создания пользователя — POST. Потом форму редактирования — тоже POST. Потом кнопку «удалить» — снова POST. «Зачем париться, если всё работает?»

Работает — да. Но теряется семантика. Ты не можешь по URL и методу понять, что делает запрос. Кешировать нельзя. Автоматические инструменты (Swagger, API Gateway, прокси) не понимают структуру.

Таблица HTTP методов

Метод	Действие	Тело запроса?	Идемпотентный?
GET	Получить данные	Нет	Да
POST	Создать ресурс	Да	Нет
PUT	Заменить ресурс целиком	Да	Да
PATCH	Изменить часть ресурса	Да	Зависит
DELETE	Удалить ресурс	Нет	Да

Что такое идемпотентность

Идемпотентный запрос — это запрос, который можно повторить любое количество раз с одним и тем же результатом.

DELETE /users/42 два раза подряд — после первого пользователь удалён. Второй запрос тоже вернёт 204 (или 404), но состояние системы не изменится. Это идемпотентно.

POST /orders два раза — создаст два заказа. Не идемпотентно. Поэтому при retry нужно быть осторожным — используйте idempotency key.

Совет: Объясни аудитории практическую важность: браузеры и прокси могут автоматически повторить GET-запрос при ошибке сети. POST — никогда, потому что знают, что он не идемпотентный.

Статус-коды

Статус-код — это ответ сервера: «что произошло». Не текст в теле, а машиночитаемый трёхзначный код.

Код	Класс	Когда использовать
-----	-------	--------------------

200 OK	2xx	Успешный запрос, данные в теле ответа
201 Created	2xx	Ресурс создан (обычно после POST). Хорошо добавить Location-заголовок
204 No Content	2xx	Успешно, но тела нет (типично для DELETE)
400 Bad Request	4xx	Клиент прислал что-то невалидное (плохой JSON, не те поля)
401 Unauthorized	4xx	Нет или неверный токен — нужно авторизоваться
403 Forbidden	4xx	Токен есть, но прав не хватает
404 Not Found	4xx	Ресурс не найден
422 Unprocessable Entity	4xx	Структура корректная, но данные не прошли валидацию
500 Internal Server Error	5xx	Ошибка на сервере — клиент ни в чём не виноват
503 Service Unavailable	5xx	Сервер временно недоступен (деплой, перегрузка)

Главная ошибка со статус-кодами

Частая ошибка: Возвращать 200 OK с телом вида { "error": "User not found" }. Это убивает всю семантику. Клиент думает, что всё хорошо, и начинает обрабатывать ответ как успешный.

Правило простое: если что-то пошло не так — используй 4xx или 5xx. Тело ошибки может содержать детали, но статус-код должен точно отражать ситуацию.

401 vs 403 — почему джуны путают

401 Unauthorized — нет токена или токен невалидный. Буквально: «я тебя не знаю, представься».

403 Forbidden — токен есть, пользователь известен, но прав не хватает. «Я тебя знаю, но туда нельзя».

Хорошо: Пользователь не залогинен → 401

Хорошо: Пользователь залогинен, но пытается удалить чужой аккаунт → 403

Блок 4 — Проектирование запросов и ответов

10 минут

Заголовки (Headers)

Заголовки HTTP — это метаданные запроса и ответа. Тело несёт данные, заголовки — контекст.

Самые важные для REST:

- **Content-Type:** Content-Type: application/json
 - Говорит серверу, в каком формате тело запроса. Обязателен при POST/PUT/PATCH.
- **Accept:** Accept: application/json
 - Говорит серверу, какой формат ответа ожидает клиент.
- **Authorization:** Authorization: Bearer <token>
 - Передаёт токен авторизации. Подробнее — в блоке про безопасность.

Структура тела запроса

Для POST/PUT/PATCH тело — это JSON-объект с данными ресурса. Правило: отправляй только то, что нужно серверу для этой операции.

```
{
  "name": "Иван Петров",
  "email": "ivan@example.com",
  "role": "user"
}
```

Структура тела ответа

Нет единого стандарта, но есть хорошие практики. Главное — консистентность: один стиль по всему API.

Успешный ответ (один объект):

```
{
  "id": 42,
  "name": "Иван Петров",
  "email": "ivan@example.com",
  "created_at": "2024-01-15T10:30:00Z"
}
```

Ответ с ошибкой:

```
{
  "error": "validation_error",
  "message": "Email уже используется",
  "details": { "field": "email" }
}
```

```
}
```

Пагинация

Никогда не возвращай все записи без лимита — в продакшне это убьёт сервер. Стандартный подход:

```
GET /users?page=2&limit=20
```

Ответ включает метаданные пагинации:

```
{
  "data": [...],
  "total": 150,
  "page": 2,
  "limit": 20,
  "pages": 8
}
```

Консистентность — важнее красоты

Частая ошибка: Самая частая проблема в реальных API — несогласованность. В одном эндпоинте `user_id`, в другом `userId`, в третьем `userIdentifier`. Это больно для всех, кто использует API.

Выбери одно соглашение и придерживайся его везде: `snake_case` или `camelCase` для полей, ISO 8601 для дат, `id/slug` для идентификаторов.

Блок 5 — Безопасность

15 минут

Боль: «это внутренний сервис»

Классическая история: джун деплоит API на сервер, не добавляет авторизацию — «это же только для фронтенда, кто найдёт». Потом кто-то находит. Или кто-то другой на команде случайно делает запрос не туда.

Безопасность — не опция для «потом». Это часть контракта API с первого дня.

CORS — почему браузер блокирует запросы

CORS (Cross-Origin Resource Sharing) — механизм безопасности браузера. По умолчанию браузер не даёт JavaScript на сайте example.com делать запросы к api.other.com.

Почему? Чтобы вредоносный сайт не мог от имени твоего браузера (с твоими куками) делать запросы к твоему банку.

Как работает CORS

Для «сложных» запросов (POST с JSON, запросы с заголовком Authorization) браузер сначала делает preflight-запрос:

```
OPTIONS /users HTTP/1.1
Origin: https://myapp.com
Access-Control-Request-Method: POST
```

Сервер отвечает, что разрешено:

```
Access-Control-Allow-Origin: https://myapp.com
Access-Control-Allow-Methods: GET, POST, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
```

Только после этого браузер отправляет реальный запрос. Важно: CORS — это браузерная защита. curl и Postman его не проверяют — там ограничений нет.

Совет: Частая ошибка — поставить Access-Control-Allow-Origin: * и забыть. Это ок для публичных API, но для API с авторизацией нельзя: при wildcard * браузер не пропустит куки и заголовок Authorization.

Аутентификация через API Key

Самый простой способ: сервер выдаёт уникальный ключ, клиент передаёт его в каждом запросе.

```
GET /data HTTP/1.1
X-API-Key: sk_live_abc123xyz
```

Плюсы: просто реализовать и использовать. Хорошо для server-to-server интеграций, где нет пользователя.

Минусы: ключ статичный — если утёк, нужно перевыпускать. Нет встроенного срока жизни.

Bearer Token и JWT

Более современный подход — токены. Клиент логинится и получает токен. Затем передаёт его в каждом запросе:

```
GET /profile HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

Что внутри JWT

JWT (JSON Web Token) — это три части, закодированные в base64 и разделённые точками: header.payload.signature

Payload содержит claims — данные о пользователе:

```
{
  "sub": "42",           // id пользователя
  "role": "admin",       // роль
  "exp": 1735689600      // срок жизни (unix timestamp)
}
```

Сервер проверяет подпись и извлекает данные из токена — без запроса в базу. Это главное преимущество JWT: stateless авторизация.

Частая ошибка: JWT payload не зашифрован — только подписан. Любой может декодировать и прочитать payload. Не кладите туда пароли, секреты или чувствительные данные.

Совет: OAuth 2.0 — следующий уровень: делегирование авторизации ("войти через Google"). Это отдельная большая тема, выходящая за рамки вебинара.

Блок 6 — Версионирование

5 минут

Боль: сломанные клиенты

Ты поменял структуру ответа — добавил вложенный объект вместо плоских полей. Удобно! Но мобильное приложение, которое уже у пользователей на телефонах, парсит старую структуру. Оно ломается.

API — это публичный контракт. Ты не можешь просто взять и поменять его. Нужно версионирование.

Три подхода к версионированию

1. Версия в URL (самый популярный)

```
GET /v1/users
GET /v2/users
```

Плюсы: очевидно, легко тестировать, работает везде. Минусы: URL технически не должен меняться при изменении представления — пуристы REST против этого. На практике — все так делают.

2. Версия в заголовке

```
GET /users HTTP/1.1
API-Version: 2
```

Плюсы: URL чистый. Минусы: неочевидно, сложнее тестировать в браузере, легко забыть.

3. Версия через Асепт заголовков (Content Negotiation)

```
Accept: application/vnd.myapi.v2+json
```

Плюсы: «правильно» с точки зрения HTTP. Минусы: громоздко, плохо поддерживается инструментами.

Что выбрать

Для большинства проектов — версия в URL. Просто, понятно, работает. Когда выпускаешь v2 — не удаляй v1 сразу. Дай клиентам время мигрировать, установи deadline и предупреди заранее.

Совет: Хорошая практика: добавить в ответ заголовок `Deprecation: true` для старых версий, чтобы клиенты видели предупреждение в логах.

Блок 7 — Практика / Демо

10 минут

Для докладчика: Этот блок — placeholder. Выбери один из трёх форматов ниже в зависимости от аудитории и настроения.

Вариант А — Разбор реального API (рекомендую)

Открой документацию GitHub REST API (docs.github.com/en/rest) или Stripe API. Пройдись по конкретным эндпоинтам и покажи, как всё, о чём говорили, работает в реальности:

- Правильные URL с ресурсами: `/repos/{owner}/{repo}/issues`
- Правильные HTTP методы: GET для получения, POST для создания
- Красивые статус-коды: 201 при создании, 404 если нет
- Пагинация через query params: `?per_page=30&page=2`
- Авторизация: `Authorization: Bearer <token>`
- Версионирование: `Accept: application/vnd.github.v3+json`

Сделай живой запрос через curl или Postman прямо во время вебинара — это всегда работает лучше любых слайдов.

Вариант В — Антипаттерны

Покажи «плохой» API и попроси аудиторию найти проблемы:

```
POST /getAllUsers
POST /getUserById?userId=42
POST /deleteUser
POST /createNewUserAccount
```

Потом вместе исправьте — пусть аудитория сама предлагает правильные версии.

Вариант С — Мини live coding

Напиши минимальный REST API прямо во время вебинара: 3-4 эндпоинта для ресурса «задачи» (to-do list). Язык — на твой выбор, это займёт 10-15 минут.

Покажи: правильные роуты, методы, статус-коды, обработку ошибок.

Блок 8 — Итог и Q&A

5 минут

Чеклист хорошего REST API

Закончи вебинар этим чеклистом — пусть аудитория сохранит и сверяется на своих проектах:

- URL описывает ресурс (существительное), не действие
- HTTP метод несёт семантику (GET читает, POST создаёт, DELETE удаляет)
- Статус-коды точно отражают результат (не 200 OK для ошибок)
- Консистентная структура запросов и ответов по всему API
- CORS настроен корректно (не * при использовании Authorization)
- Авторизация есть — хотя бы API Key или Bearer Token
- Версионирование есть — хотя бы /v1/ в URL
- Пагинация есть — не возвращаешь все записи сразу

Что дальше

REST — это фундамент. Когда освоишь его — изучи:

- OpenAPI / Swagger — как документировать API
- REST vs GraphQL — когда гибкость запросов важнее простоты
- REST vs gRPC — когда важна скорость и строгая типизация
- OAuth 2.0 — делегированная авторизация (войти через Google/GitHub)
- Rate limiting и throttling — защита API от перегрузки

Q&A

Оставь 5 минут на вопросы. Хорошие вопросы, которые часто задают:

- «Когда использовать PUT, а когда PATCH?» — PUT заменяет весь ресурс, PATCH меняет только указанные поля
- «Как правильно удалять — DELETE или POST /archive?» — зависит от бизнес-логики: физическое удаление vs мягкое
- «Нужен ли REST для внутренних микросервисов?» — не обязательно, gRPC часто лучше подходит
- «Как обрабатывать долгие операции?» — возвращай 202 Accepted + Location с эндпоинтом для проверки статуса

Удачного вебинара!